AI assist week -13.4

T. VIKAS

2303A52281

BATCH 43

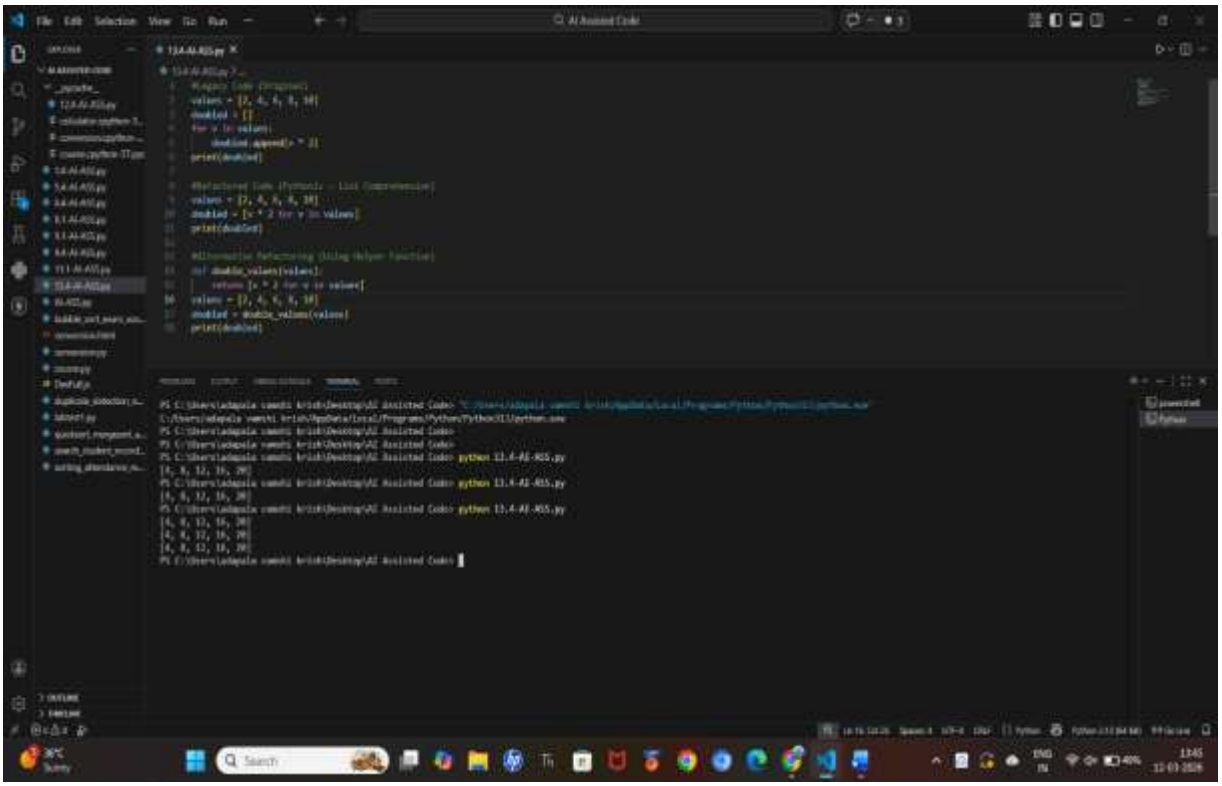
TASK-1

Scenario: (Refactoring Data Transformation Logic)

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Prompt: Refactor the following Python code to make it more Pythonic and readable using list comprehensions while keeping the same output.

Code:



Observation:

The AI suggested using list comprehension to replace the explicit loop.

This reduced the number of lines, improved readability, and made the code more Pythonic while maintaining the same output.

TASK-2

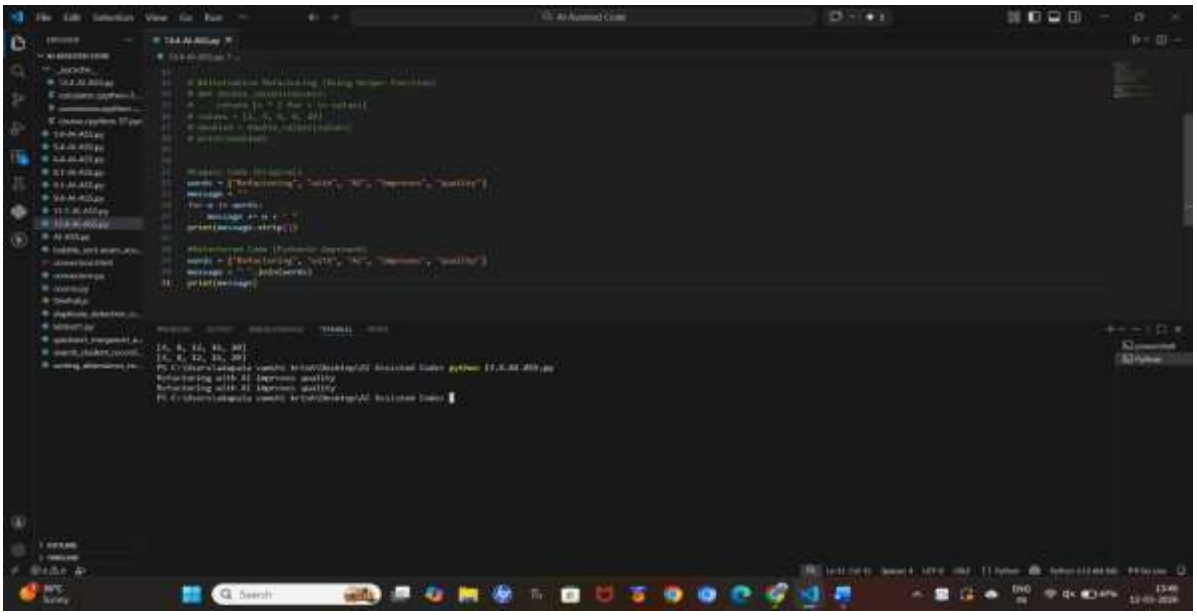
Scenario: (Improving Text Processing Code Readability)

You are working on a log message generator that builds messages word by word.

Prompt:

Improve the following Python code that constructs a sentence using repeated string concatenation. Suggest a more efficient and readable approach.

Code:



Observation:

The AI recommended using the join() method to combine the words into a sentence.

This approach avoids repeated string concatenation, improves performance, and makes the code more concise and readable.

TASK-3

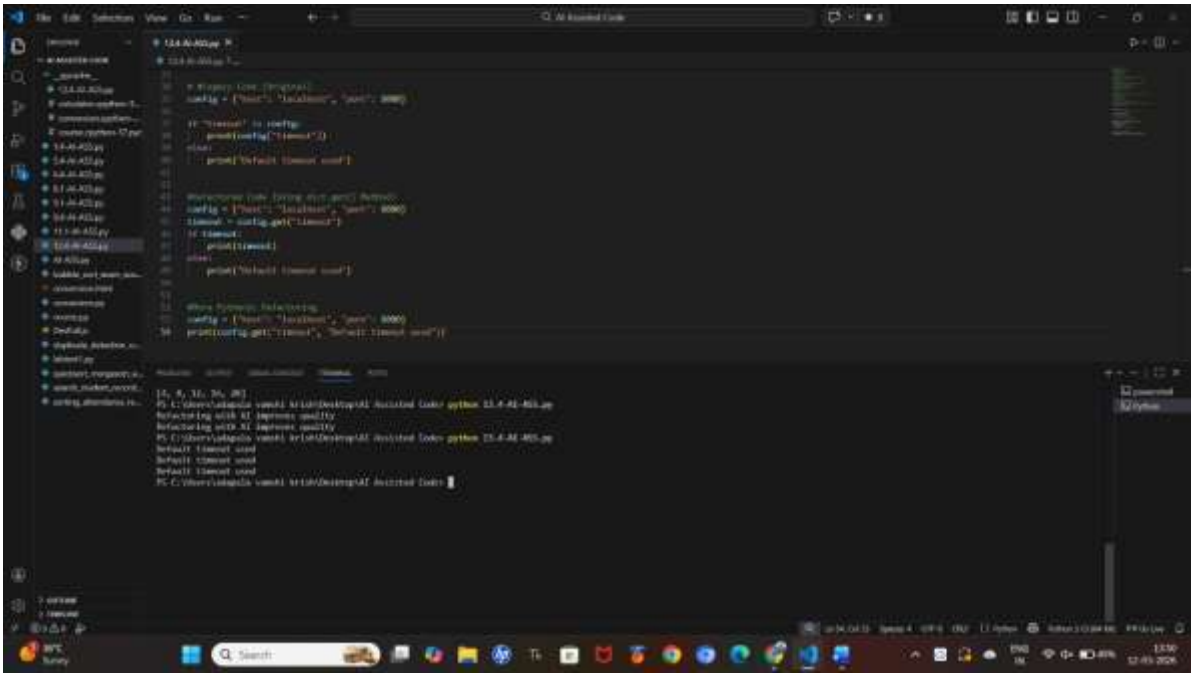
Scenario: (Safer Access to Configuration Data)

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Prompt:

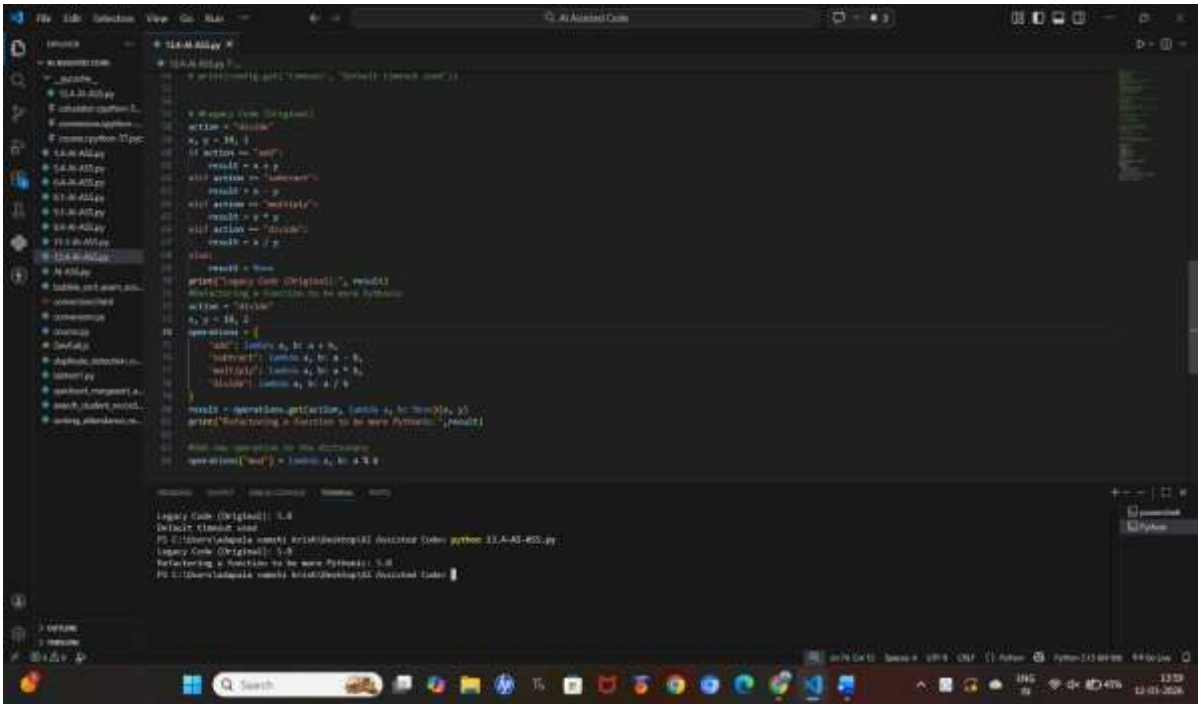
Refactor the following Python code to safely access dictionary keys using a more Pythonic method while preserving the behavior for missing keys.

Code:



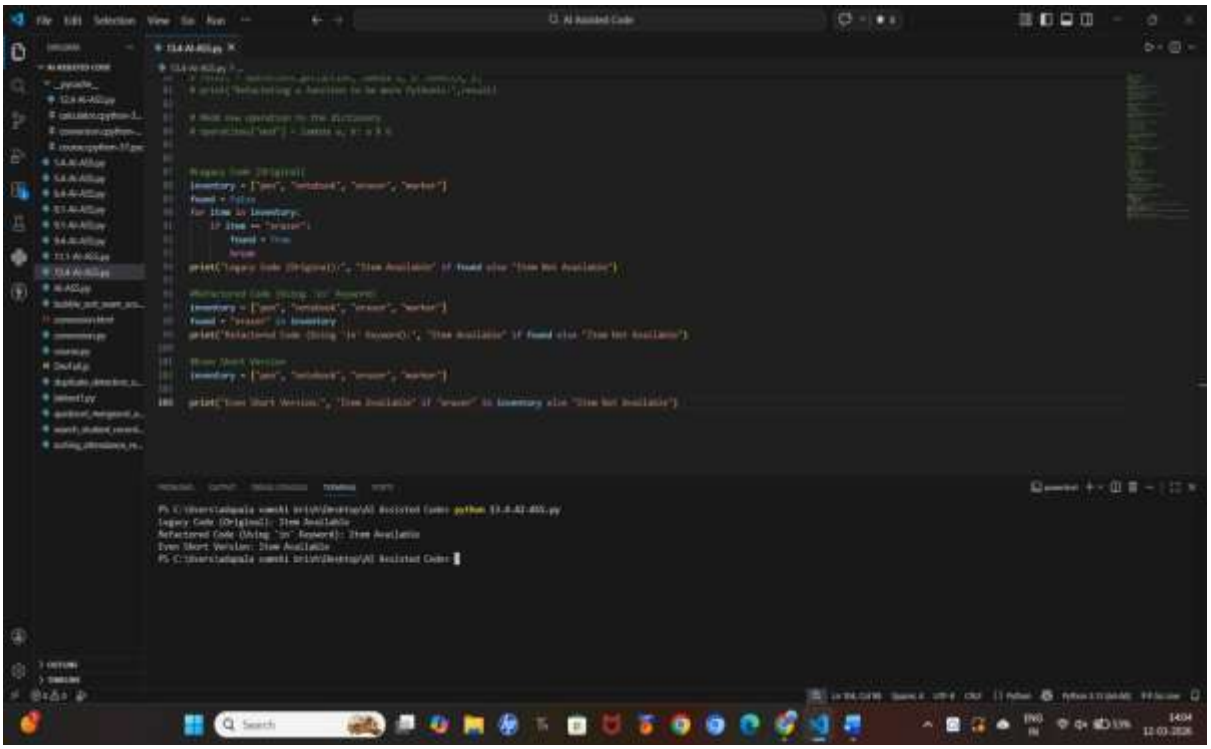
Observation:
The AI suggested using the dictionary get() method to safely retrieve values.
This simplifies the code and prevents errors when a key is missing while keeping the same behavior.

TASK-4
Scenario: (Refactoring Conditional Logic for Scalability)
You are enhancing a utility module that performs different operations based on user input.
Prompt: Refactor the following Python code that uses multiple if-elif conditions. Suggest a cleaner and scalable approach using mapping techniques.
Code:



Observation:
The AI recommended replacing the multiple **if–elif conditions** with a **dictionary mapping of operations**.
This makes the code easier to maintain, scalable, and cleaner when adding new operations.

TASK-5
Scenario: (Simplifying Search Logic in Collections)
You are reviewing legacy inventory-check code that uses manual loops to find elements.
Prompt: Refactor the following Python code that searches for an item in a list using a loop. Suggest a more concise and readable Pythonic approach.
Code:



Observation:

The AI suggested using the in operator to check if the item exists in the list.
This eliminates the manual loop, simplifies the logic, and improves readability.